Mid-Sem Exam: CSE 202, Winter Session 2026

Marks: 40

Time: 1 hr

Note: (1) Please use the same notations used in the class for E-R diagram (rectangle for Entity Set, diamond for relationship set, oval for attribute, double rectangle for weak entity, → for at most one mapping, --- for Zero or more mapping, double line for total participation, etc). (2) Schemas given in Question 5 should be used in questions 8-10.

Here is a part of the case-study of "Wow Stylish Saloon Pvt Ltd.

Wow Stylish Saloon operates multiple saloon branches across Delhi, where each branch is uniquely identified and maintains its own location details. The saloon employs staff who may act as managers or non-managers. An employee may work in zero or more branches; however, each branch can have at most one non-manager assigned to it. Branch management follows a one-to-one relationship: each branch is managed by at most one manager, and each manager manages at most one branch. Clients are uniquely identified and may visit multiple branches over time. **Each saloon branch offers various services such as haircuts and facials, each with a defined price, but not all branches necessarily offer all available services.** Employees are restricted to providing at most one type of service, while clients may request multiple services where the price of each service is fixed, potentially from different employees. For each service transaction, the client selects the employee who provides the service at a specific branch on a given date. This structure ensures clear separation of responsibilities, accurate tracking of service delivery, and enforcement of operational constraints across branches.

Question 1: Draw the E-R diagram of the highlighted text in the above requirements. Show the primary key of each entity and relationship sets, and write the cardinality and participation constraints, if any. (1+1+1+1)

Question 2: Repeat the question 1 sub-tasks for the underlined text. (2+1+1+1)

Question 3: Is there any Weak entity set in the above case-study? If Yes, which one and why? If No, why it can't be? (2)

Question 4: Convert the E-R diagram of Question 2 into the relational schemas. Write the Primary key and Foreign key of each relation, if there is any. (2+1)

Question 5: Let us assume Ms Jaya Kumari comes up with the following (some of the) relational schemas of the above case-study. The Primary keys are underlined. (2*3)

Employee(EmpID, Name, Role, City), ['Role' has two values Mgr or non-Mgr]

Client(ClientID, Name, City) **Branch**(B#, City, S#, SName) 1 → ?

Provides(EmpID, ClientID, S#, ServiceDate)

2 or more Services

γ , Sum (

2 or more? ⇝ All - [illegible] 1

# Servics	Serv Date
3	

Total Services

Total # Services Provided by. Service date.

Aggregate Fn?

CNT( )

Select any three queries given below and write the Relational Algebraic Expression (RAE) on these schemas. You are not allowed to use the aggregate operator, unless stated.

A. How many clients are served on each day [use aggregate operator]?
B. List all client names along with their clientID from "Kalkaji' city.
C. List all clients who have taken two or more services from employees.
D. List all client IDs who have taken multiple services from the same employee.
E. List all those clients who have taken all services provided by 'Kalkaji' Branch.
F. List the city that has only one employee from that city.
G. List all employees who have not provided any service to any client.

Question 6: Select any three queries excluding those selected in question 5, and write the SQL queries. Here you can use the aggregate operator. (2*3)

Question 7: Mr Ravi is designing the relational schemas of the above application. One of the relational schema is given below, which is an un-normalized relation,

A B C D E

Emp-BranchService(EID, EmpName, BranchID, BranchCity, ServiceTransactionID, ServiceID, ServicePrice, ServiceDuration)

F G H

Here the functional dependencies are:

EmpID → EmpName

BranchID → BranchCity

ServiceTransactionID → ServiceID

$A \rightarrow B$

$C \rightarrow D$

$E \rightarrow F$

$\{A, G, E\}$

(a) What are the candidate keys in this un-normalized relation? (2)
(b) Normalize the Emp-BranchService relation into a lossless and dependency preserving relations, and indicate the primary key of each normalized relation and the highest normal form. (2+1+1)

Question 8: An index table **Client-Index** is created on the primary key of the above relation **Client.** This index table resides in the main memory. Write two valid SQL queries that the query processor will answer them using this index table. (2*2)

Question 9: Write an SQL for creating a view **(Mumbai-Branch)** on the above **Branch** relation which provides services offered by the branches of 'Mumbai' city. (2)

Question 10: Write an SQL query which analyzes the total number of services provided by ServiceDate. (2)

2 or more - Service

A → B

Final Exam: CSE 202, Winter Session 2026

Marks: 30 **Time: 2 hr**

Note: Read the instructions carefully. Several questions are based on the case study given below:

1. **All questions should be answered in the same order (within each Section) as given in the paper.**
2. Section-A has 30 questions. Each question carries 0.5 marks. The answer of each question is only a few words (not more than half a line or a simple calculation or an SQL or an RAE).
3. Section-B and Section –C has 7 and 8 questions, respectively. Each question carries 1.0 mark.
4. There is no partial marking in any questions of Sections A, B and C.

Wow Stylish Saloon – Database Design Specification

Wow Stylish Saloon operates a network of salon branches across Delhi, offering a variety of grooming services such as haircuts, facials, and other personal care treatments. The organization requires a centralized database system to manage its distributed operations efficiently while ensuring data consistency, integrity, and scalability. Each salon branch is uniquely identified and maintains its own location details, including address, locality, and city. While branches operate independently in terms of service price and delivery, they are logically integrated through the centralized database system, which supports reporting, operational monitoring, and decision-making.

The salon employs a set of staff members who may function either as managers or non-managerial employees. Each employee is uniquely identified and may be associated with zero or more branches. However, a key operational constraint is that each branch can have at most five non-manager assigned to it. Furthermore, the management structure follows a strict one-to-one relationship, that is, each branch is managed by at most one manager, and each manager is responsible for managing at most one branch. This ensures a clear and non-overlapping managerial hierarchy across all branches.

From a service perspective, **Wow Stylish Saloon offers a predefined set of services, each identified uniquely and associated with a fixed price per branch. Not all branches offer all services; instead, each branch selectively provides a subset of services depending on demand and specialization.** Employees are restricted to providing at most one type of service, ensuring specialization and maintaining service quality. Clients, on the other hand, are uniquely identified and may visit multiple branches over time. During a visit, a client may request multiple services, potentially provided by different employees. Each service transaction is recorded with complete details, including the client, the employee providing the service, the branch where the service is delivered, and the date of the transaction. This design allows accurate tracking of service delivery and supports detailed historical analysis.

Q-8
E-R diagr

To support efficient data storage and retrieval, the system is assumed to operate on a disk-based architecture with defined physical characteristics. The disk system consists of six platters, each with two recording surfaces, resulting in a total of twelve surfaces. Each surface contains one thousand tracks. Each track is divided into two hundred sectors, with each sector storing 512 bytes of data. This results in a track size of approximately 102,400 bytes (or roughly 100 KB). The database system uses a fixed block (page) size of 4096 bytes (4 KB), allowing approximately 25 blocks per track. Across all surfaces and tracks, the disk can store approximately 300,000 blocks, yielding a total storage capacity of about 1.2 GB.

The main memory available to the database system is organized as a buffer pool consisting of 1000 pages, each of size 4 KB, resulting in a total buffer size of approximately 4 MB. This buffer pool is used to temporarily hold disk pages during query processing and transaction execution. Data is always transferred between disk and memory in units of pages, and any processing, including query execution and sorting, is performed in memory after pages are loaded into the buffer.

Records within the database are stored in a slotted page format, and each record is identified using a Record Identifier (RID), which consists of a page number (4 bytes) and a slot number (2 bytes), totalling 6 bytes. Pointers within the system include page pointers (4 bytes) and record pointers (6 bytes). For example, an employee record containing attributes such as employee ID, name, role, salary, age, and service identifier occupies approximately 45 bytes. Given a block size of 4096 bytes, the page header size is 100 bytes and slot entry size (offset) is 4 bytes, this results in a blocking factor of roughly 81 records per page, which influences storage utilization and I/O performance.

The database is expected to handle a workload characterized by frequent insert operations, particularly for recording service transactions, along with moderate read operations for reporting and querying purposes. Common queries include retrieving the service history of a client, computing revenue generated by a branch over a specified period, analyzing employee performance, listing services offered at a branch, and identifying the manager responsible for a branch. These queries involve a mix of equality conditions, range predicates (especially on dates), and join operations across multiple relations such as transactions, employees, services, and branches.

To support this workload efficiently, appropriate indexing strategies are required. Primary indexes are maintained on key identifiers such as B#, E#, and S#. Secondary indexes are created on frequently queried attributes such as C#, B#, and transaction dates. Composite indexes, such as those on (B#, Date) and (E#, Date), are used to optimize multi-attribute queries. B+ tree indexes are preferred for range queries, while hash-based indexing may be considered for equality-based lookups. Additionally, clustering of transaction data based on date or branch can significantly reduce I/O costs for range queries and reporting operations.

In the *Wow Stylish Saloon* system, multiple transactions occur concurrently—for example, booking a service, assigning an employee, updating service availability, and processing payments across different branches. When several clients simultaneously request overlapping services (e.g., the same stylist at the same time), or when managers update employee assignments while bookings are being made, **concurrency issues** such as lost updates, dirty reads, and inconsistent schedules can arise. Without proper control, one transaction may overwrite another's updates (e.g., double-booking a stylist), or read uncommitted data (e.g., viewing a service slot that is later rolled back). To ensure correctness, the system must enforce **serializability using protocols like Two-Phase Locking (2PL)** and preferably **Strict 2PL** to avoid cascading rollbacks. Additionally, failures such as system crashes during transaction execution require robust **recovery mechanisms (logging, checkpoints, undo/redo)** to maintain durability and atomicity, ensuring that partially completed bookings or payments do not corrupt the database state.

Overall, this system integrates business logic, logical data modelling, and physical storage considerations into a unified design. The specification ensures that the database system is capable of supporting operational requirements, enforcing constraints, and delivering high performance through careful consideration of disk layout, memory management, and indexing strategies. This comprehensive design provides a strong foundation for implementing a scalable and efficient database solution for Wow Stylish Saloon.

Some of the relations schema are as follows. Let the schema of **EMP** be (E#, Name, Role, B#, S#), **BRANCH** be (B#, Name, City), **SERVICE** be (S#, B#, Name, Price). **CLIENT** be (C#, Name, City), **TRANSACTION** be (T#, C#, E#, S#, B#, Date) The index is created on the Primary Key of each relation.

Section-A

1. What is the cardinality between Branch and Manager?
2. Is the participation of Branch in " managed by" relationship total or partial?
3. Write Relational Algebraic Expression (RAE) for: "Find all client IDs who visited branch B101."
4. Which relational algebraic operator is used for combining Employee and Service?
5. Write SQL to find all services priced above 500.
6. Write SQL to count number of transactions per branch.
7. If S#, B# → Price, is the relation SERVICE in 3NF?
8. Draw the complete E-R diagram of the highlighted text. (with cardinality Constraints & Participation constraints)
9. What is BCNF condition?
10. If the block size of EMP relation is 4096 bytes, the page header size is 100 bytes and slot entry size (offset) is 4 bytes, this results in a blocking factor of roughly 81 records per page if we use slotted page structure. How?
11. What does the (Page#, Slot#) indicate in the Index file?
12. What is the unit of data transfer between disk and memory?
13. Which index should be created to efficiently answer "Find all transactions between two dates"? Name the attributes as well as the index type.
14. Which index type is best for query with multiple conditions on Role, B#, Service_Name?
15. What does a covering index eliminate?
16. Which join algorithm is best for: Transaction ⋈ Service (large tables) [Natural Join] if we have very low main memory available, say only 4 buffers?
17. When will the query optimizer prefer full table scan in the case of selection query?
18. If **EMP** relation has B+ tree index, what is the cost of index lookup with tree height = 3?
19. What property ensures all-or-nothing execution of a transaction?
20. Which composite index helps query "Who provided the maximum number of services in last month"?
21. What is the purpose of a log file? Answer in the context of ACID properties.
22. How many cylinders are there on the disk?
23. Explain (in both cases, Yes or No) whether a covering index can be used to answer the query on a relation (B#, S#, Price) – "What is the average price of service in each Branch"?
24. Is a composite index on (E#, Date) more effective than indexes on individual attributes for a query "Find all employees who have provided services from 19 Jan 2026 to 20 April 2026?
25. Which one takes fewer I/Os; clustered index scan or unclustered index lookup, for a range query with very low selectivity and why?
26. Why does the buffer size affect the join operations cost for block nested loop join algorithm?
27. Given: T1: W(A), T2: W(A), T1: Commit, T2: Commit. Is the schedule strict?
28. Given: T1: R(A), T2: W(A), T1: W(A), T2: Commit, T1: Commit Is the schedule conflict-serializable?
29. If a schedule follows **Strict 2PL**, which of the following is guaranteed?
    (a) Serializability (b) Cascadeless (c) Recoverability (d) All of the above
30. Given a schedule with a dirty read, can it be cascadeless?

Section-B

I. Write an SQL query for -- Retrieve the details of all employees who provide the service "Haircut".
II. For the query - Retrieve the details of all employees who provide the service "Haircut", what indexes (that is, primary/secondary/bitmap index on <relation-name(attribute(s))> will be recommended for most efficient query execution?
III. Write a relational algebraic expression for -- Find all transactions performed between two given dates.
IV. Write an SQL for ---Find all transactions where service is "Facial" and branch is "B102" and employee role is "Non-manager".

V. Write a relational algebraic expression for --- Find employees who have served more than 50 clients in the last month (i.e. March 1, 2026 to March 31, 2026). Also which index type will be recommended here to create on which relation(attribute) and why (give the reason)?

VI. Write the SQL query for the above question V.

VII. We need to fetch the employee information for a given S#. Explain how the join between Employee and Service (being S# common in both relations) will be executed if an index exists on Employee(S#). That is, which join algorithm will be used and how will it work?

Section - C

Since the 'Wow Stylish Saloon' operates multiple branches where clients book services (haircut, facial, etc.), employees provide services, payments and service records are stored in the **Transaction table** and each transaction updates (a) branch revenue [Data item A] (b) employee workload [Data item B] and (c) client service history [Data item C]. Let us consider the following simplified database schemas:

Employee(E#, S#, WorkCount)
Client(C#)
Branch(B#, Date, Revenue)
Transaction(T#, C#, E#, B#, Amount)

Data items which are being modified by the transactions are Revenue, WorkCount, and Client History.

T1 (Service booking transaction): Read(A), A = A + 500, Write(A), Read(B), B = B + 1, Write(B)

T2 (Discount Adjustment Transaction): Read(A), A = A – 100, Write(A)

T3 (Client History Update Transaction): Read(C), C = C + 1, Write(C)

T4 (Audit Transaction): Read(A), Read(B)

Questions:

(1) Let the Schedule S1 be T1: Read(A), T2: Read(A), T1: Write(A), T2: Write(A). Is this schedule S1 serializable? If yes, why. If not, why? What anomaly exists here?

(2) In the schedule S2, T1: Read(A), T1: A = A + 500, T1: Write(A), T2: Read(A), T2: Commit, T1: Abort. What anomaly does occur here? How to prevent the anomaly? Also is S2 recoverable? If yes or no, why?

(3) Let S3 be: T1: Read(A), T2: Read(A), T1: A = A + 500, T2: A = A – 100, T1: Write(A), T2: Write(A). What anomaly occurs here and what would be the value of A, assuming the initial value of A is 1000?

(4) Let the schedule S4 be T1: Read(A), T2: Read(A), T2: Write(A), T1: Read(A). What anomaly occurs here and why does it happen?

(5) In the following Schedule S5, what is the problem in this schedule S5 and how to resolve it?

T1: Lock(A)
T2: Lock(B)
T1: Lock(B) (waits)
T2: Lock(A) (waits)

(6) Is the following Schedule S6 recoverable or not? Justify your answer.
T1: Write(A), T2: Read(A), T2: Commit, T1: Abort

(7) Is this Schedule S7 serial [redacted]e? Why? T1: Read(A), T1: Write(A), T2: Read(A), T2: Write(A)

(8) Let the Schedule S8 be: T1: R(A), T2: R(A), T3: W(A), T1: W(A), T2: W(A), T3: Commit, T1: Commit, T2: Commit. Is this schedule serializable? What anomalies exist here? Is this schedule recoverable?